Manual do Desenvolvedor

IgnisGeo — Sistema Web de Priorização Geoespacial de Áreas de Risco de Queimadas

V1.2026

Guia completo de configuração e execução do ambiente de desenvolvimento local em Ubuntu / Linux Mint.

Cobertura:

- ✓ Git clone e configuração inicial
- ✓ Pré-requisitos: Docker, Python, Node.js
- ✓ Variáveis de ambiente (.env)
- ✓ Docker Compose: 5 containers
- ✓ Migrações e banco de dados PostGIS
- ✓ Exportação do CSV do INPE BDQueimadas
- ✓ Importação via endpoint da API
- ✓ Execução do TOPSIS Fuzzy
- ✓ Testes automatizados (46 testes)
- ✓ Validação dos endpoints com curl
- ✓ Documentação pdoc
- ✓ Comandos de manutenção

Rogério Cerqueira
Engenharia de Computação — UEFS · TCC 2026.1

Repositório: `https://github.com/rogeriocerqueira/ignisGeo`

Sumário

1 Pré-requisitos **3**
1.1 Verificar versões instaladas . . . 3
1.2 Instalar Docker e Docker Compose V2 . . . 3
1.3 Instalar Python 3.12 e pip . . . 4
1.4 Instalar Node.js 20 via nvm . . . 4

2 Git Clone e Configuração Inicial **4**

3 Ambiente Virtual Python (backend local) **5**

4 Docker Compose — Subindo os Containers **6**
4.1 Comandos Docker essenciais . . . 7

5 Banco de Dados — Migrações e PostGIS **7**
5.1 Verificar extensão PostGIS . . . 8
5.2 Verificar tabelas criadas . . . 8

6 Exportação do CSV do INPE BDQueimadas **8**
6.1 Estrutura do CSV do INPE . . . 9
6.2 Script de preparação do CSV . . . 9

7 Importação dos Dados via API **9**

8 Execução do TOPSIS Fuzzy **11**
8.1 TOPSIS com filtros . . . 11

9 Validação dos Endpoints com curl **12**
9.1 GET /api/ranking/ — Ranking paginado . . . 12
9.2 GET /api/focos/geojson/ — GeoJSON dos focos . . . 13
9.3 GET /api/serie-temporal/ — Série mensal . . . 13
9.4 GET /api/graficos-dados/ — Dados dos gráficos . . . 13
9.5 GET /api/correlacao/ — Matriz de correlação . . . 13
9.6 GET /api/areas-risco/geojson/ — GeoJSON das áreas . . . 14

10 Frontend — Vue 3 **14**
10.1 Acessar a interface . . . 14
10.2 Desenvolvimento frontend local (sem Docker) . . . 14
10.3 Build de produção . . . 14

11 Testes Automatizados **15**
11.1 Testes com cobertura de código . . . 15

12 Documentação pdoc **16**

13 Comandos de Manutenção **16**
13.1 Banco de dados . . . 16
13.2 Git — Fluxo de trabalho . . . 17

13.3 Limpeza do ambiente Docker 17

14 Resolução de Problemas Comuns **18**
14.1 Container `db` não inicia 18
14.2 PostGIS não disponível 18
14.3 Erro de importação circular no Django (`test.py`) 18
14.4 Frontend não conecta ao backend 18
14.5 Celery worker não processa tasks 19

15 Checklist Final — Sistema Rodando **19**

16 Resumo dos Endereços Locais **20**

1. Pré-requisitos

Antes de clonar o projeto, o ambiente deve ter as ferramentas abaixo instaladas. Todos os comandos foram validados em Ubuntu 22.04 LTS e Linux Mint 21.

1.1. Verificar versões instaladas

```
# Verifica cada ferramenta necessaria
git    --version   # >= 2.34
docker --version   # >= 24.0
docker compose version  # >= 2.20  (sem hifem - V2)
python3 --version  # >= 3.10
node   --version   # >= 18.0
npm    --version   # >= 9.0
curl   --version   # >= 7.80
```

1.2. Instalar Docker e Docker Compose V2

```
# Remove versoes antigas se houver
sudo apt remove docker docker-engine docker.io containerd runc

# Adiciona repositorio oficial Docker
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli \
     containerd.io docker-compose-plugin

# Adiciona usuario ao grupo docker (sem sudo em cada comando)
sudo usermod -aG docker $USER
newgrp docker

# Verifica instalacao
docker --version
docker compose version
```

Esperado:
Docker version 24.x.x
Docker Compose version v2.x.x

1.3. Instalar Python 3.12 e pip

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3-pip
python3 --version   # Python 3.12.x
```

1.4. Instalar Node.js 20 via nvm

```
# Instala nvm (Node Version Manager)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/
   install.sh \
  | bash
source ~/.bashrc

# Instala e usa Node 20 LTS
nvm install 20
nvm use 20
node --version   # v20.x.x
```

2. Git Clone e Configuração Inicial

Passo 1 — Clone do repositório

```
# Clona o repositorio principal
git clone https://github.com/rogeriocerqueira/ignisGeo.git

# Entra no diretorio raiz do projeto
cd ignisGeo

# Visualiza a estrutura do projeto
ls -la
```

Estrutura esperada:

```
drwxr-xr-x  backend/        # Django + DRF + Celery
drwxr-xr-x  frontend/       # Vue 3 + Vite
drwxr-xr-x  docs/           # Diagramas e documentacao
-rw-r--r--  docker-compose.yml
-rw-r--r--  .gitignore
-rw-r--r--  README.md
```

Passo 2 — Criar arquivo de variáveis de ambiente

O arquivo `.env` nunca é versionado (está no `.gitignore`). Crie a partir do template:

```
# Copia o template de exemplo
cp .env.example .env

# Edita com suas configuracoes locais
nano .env
```

Conteúdo do `.env`:

```
# Banco de dados
POSTGRES_DB=queimadas
POSTGRES_USER=postgres
POSTGRES_PASSWORD=ignisGeo@2026
DB_HOST=db
DB_PORT=5432

# Django
DJANGO_SECRET_KEY=troque-por-uma-chave-secreta-forte-aqui
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# Celery
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/0

# Frontend
VITE_API_BASE_URL=http://localhost:8000/api
```

Atenção: nunca commite o arquivo `.env` no Git. Ele contém credenciais e está protegido pelo `.gitignore`. O GitHub bloqueia o push se detectar tokens ou senhas no histórico.

3. Ambiente Virtual Python (backend local)

Passo 3 — Criar e ativar o ambiente virtual

```
cd backend

# Cria o ambiente virtual
python3 -m venv .venv

# Ativa o ambiente virtual
source .venv/bin/activate

# Verifica que o Python correto esta ativo
which python
# Esperado: .../ignisGeo/backend/.venv/bin/python
```

```
# Instala as dependencias do projeto
pip install -r requirements.txt
```

Dependências principais instaladas:

```
Django==5.x
djangorestframework==3.x
djangorestframework-gis==1.x
psycopg2-binary==2.x
celery==5.x
redis==5.x
gdal==3.x        # Necessario para GeoDjango
```

GDAL no Ubuntu: se `pip install gdal` falhar, instale a biblioteca do sistema antes:

```
sudo apt install -y gdal-bin libgdal-dev
export CPLUS_INCLUDE_PATH=/usr/include/gdal
export C_INCLUDE_PATH=/usr/include/gdal
pip install gdal==$(gdal-config --version)
```

4. Docker Compose — Subindo os Containers

Passo 4 — Build e inicialização dos containers

```
# A partir da raiz do projeto (ignisGeo/)
cd ..

# Build de todos os containers (primeira vez, baixa as
   imagens)
docker compose build

# Sobe todos os 5 containers em background
docker compose up -d

# Acompanha os logs em tempo real (Ctrl+C para sair)
docker compose logs -f
```

Passo 5 — Verificar containers em execução

```
docker compose ps
```

Saída esperada:

```
NAME                    SERVICE     STATUS     PORTS
ignisgeo-backend-1      backend     running
   0.0.0.0:8000->8000/tcp
```

```
ignisgeo-frontend-1     frontend    running
   0.0.0.0:5173->5173/tcp
ignisgeo-db-1           db          running
   0.0.0.0:5433->5432/tcp
ignisgeo-redis-1        redis       running
   0.0.0.0:6379->6379/tcp
ignisgeo-worker-1       worker      running
```

Todos os 5 containers devem estar com STATUS = `running`.
Se algum aparecer `exited`, verifique os logs: `docker compose logs <nome-do-servico>`

4.1. Comandos Docker essenciais

```
# Para todos os containers
docker compose down

# Para e remove volumes (APAGA o banco)
docker compose down -v

# Reinicia apenas o backend
docker compose restart backend

# Acessa o shell dentro do container backend
docker compose exec backend bash

# Acessa o psql dentro do container do banco
docker compose exec db psql -U postgres -d queimadas

# Ver logs de um container especifico
docker compose logs backend -f
docker compose logs worker  -f
docker compose logs db      -f

# Rebuild de um servico apos mudancas no codigo
docker compose up -d --build backend
```

5. Banco de Dados — Migrações e PostGIS

Passo 6 — Aplicar migrações Django

```
# Executa as migracoes dentro do container backend
docker compose exec backend python manage.py migrate

# Saida esperada (trecho):
# Applying queimadas.0001_initial... OK
# Applying queimadas.0002_alter_arearisco_geometria... OK
```

```
# Applying queimadas.0003_rename_ndvi_medio... OK
```

5.1. Verificar extensão PostGIS

```
# Acessa o psql e verifica o PostGIS
docker compose exec db psql -U postgres -d queimadas -c \
  "SELECT postgis_version();"

# Saida esperada:
# postgis_version
# ----------------
# 3.4 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
```

5.2. Verificar tabelas criadas

```
docker compose exec db psql -U postgres -d queimadas -c "\dt"

# Saida esperada:
# Schema |          Name           | Type  |  Owner
# -------+-------------------------+-------+----------
# public | queimadas_arearisco     | table | postgres
# public | queimadas_focoqueimada  | table | postgres
# public | spatial_ref_sys         | table | postgres
```

Passo 7 — Criar superusuário Django (opcional)

```
docker compose exec backend python manage.py createsuperuser
# Username: admin
# Email: admin@ignisGeo.local
# Password: (escolha uma senha)

# Acessa o admin em: http://localhost:8000/admin/
```

6. Exportação do CSV do INPE BDQueimadas

Passo 8 — Exportar dados do BDQueimadas

Acesse o portal do INPE em: `https://queimadas.dgi.inpe.br/queimadas/bdqueimadas`

Parâmetros de exportação utilizados no IgnisGeo:

Parâmetro	Valor
País	Brasil
Estados	Todos (26 estados + DF)
Biomas	Todos
Período início	01/01/2025
Período fim	30/04/2026
Formato	CSV
Satélite ref.	Aqua Tarde (padrão INPE)

O arquivo baixado terá um nome similar a: `focos_br_2025-01-01_2026-04-30.csv`

6.1. Estrutura do CSV do INPE

```
# Visualiza as primeiras linhas do CSV
head -3 focos_br_2025-01-01_2026-04-30.csv
```

```
id,latitude,longitude,data_hora_gmt,satelite,municipio,estado,
pais,municipio_id,bioma,frp,risco_fogo,numero_dias_sem_chuva,
precipitacao

1,-10.5921,-45.1234,2025-01-01 13:24:00,Aqua_Tarde,CORURIPE,
AL,Brasil,2702355,CAATINGA,45.2,0.8,15,0.0
```

6.2. Script de preparação do CSV

```
# Copia o CSV para dentro do container backend
docker compose cp \
  focos_br_2025-01-01_2026-04-30.csv \
  backend:/app/data/focos_mensal.csv

# Verifica que o arquivo chegou
docker compose exec backend ls -lh /app/data/
```

7. Importação dos Dados via API

Passo 9 — Disparar importação via endpoint

```
# Dispara a importacao assincrona via Celery
curl -s -X POST http://localhost:8000/api/importar-csv/ \
  -H "Content-Type: application/json" \
  -d '{"caminho": "/app/data/focos_mensal.csv"}' \
  | python3 -m json.tool
```

Resposta esperada:

```
{
    "mensagem": "Importacao iniciada.",
    "task_id": "abc123-def456-...",
    "caminho": "/app/data/focos_mensal.csv"
}
```

Passo 10 — Monitorar o progresso da importação

```
# Acompanha o worker em tempo real
docker compose logs worker -f

# Saida esperada (trecho):
# [2026-05-07 13:00:00,000: INFO/MainProcess]
#   Task queimadas.tasks.importar_csv_inpe[abc123] received
# [2026-05-07 13:03:22,000: INFO/ForkPoolWorker-1]
#   Task queimadas.tasks.importar_csv_inpe[abc123] succeeded
#   Result: {'importados': 3876956, 'erros': 0}
```

A importação completa leva entre 3 e 8 minutos para 3,8 milhões de registros, dependendo do hardware.

Passo 11 — Verificar total de focos importados

```
curl -s http://localhost:8000/api/estatisticas/ \
  | python3 -m json.tool
```

Resposta esperada:

```
{
    "total_focos": 3876956,
    "areas_criticas": 0,
    "areas_alto_risco": 0,
    "por_bioma": [
        {"bioma": "CERRADO",    "total": 1849819,
         "frp_media": 36.61},
        {"bioma": "AMAZONIA",   "total": 1202671,
         "frp_media": 40.09},
        {"bioma": "CAATINGA",   "total": 496033,
         "frp_media": 27.61},
        {"bioma": "MATA_ATLANTICA","total": 275140,
         "frp_media": 15.91},
        {"bioma": "PANTANAL",   "total": 39222,
         "frp_media": 48.97},
        {"bioma": "PAMPA",      "total": 14071,
         "frp_media": 14.16}
    ]
}
```

8. Execução do TOPSIS Fuzzy

Passo 12 — Executar o pipeline TOPSIS

```
# Executa o TOPSIS sobre todos os dados (sem filtros)
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
  -H "Content-Type: application/json" \
  -d '{}' \
  | python3 -m json.tool
```

Resposta esperada:

```
{
    "mensagem": "TOPSIS Fuzzy calculado para 6195 areas.",
    "periodo": {
        "inicio": "2025-01-01",
        "fim": "2026-04-30"
    },
    "filtros": {
        "estado": "Todos",
        "bioma": "Todos"
    },
    "areas_atualizadas": 6195,
    "top_5": [
        {"nome": "BARRA/BA/CERRADO",
         "score_topsis": 0.5209,
         "nivel_risco": "CRITICO",
         "ranking": 1},
        {"nome": "LUCENA/PB/MATA_ATLANTICA",
         "score_topsis": 0.4553,
         "nivel_risco": "CRITICO",
         "ranking": 2},
        {"nome": "CORURIPE/AL/CAATINGA",
         "score_topsis": 0.4161,
         "nivel_risco": "CRITICO",
         "ranking": 3}
    ]
}
```

8.1. TOPSIS com filtros

```
# Executa apenas para o estado da Bahia no Cerrado
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
  -H "Content-Type: application/json" \
  -d '{
    "estado": "BA",
    "bioma": "CERRADO",
    "data_inicio": "2025-06-01",
    "data_fim": "2025-10-31"
  }' \
  | python3 -m json.tool

# Executa apenas para 2026
```

```
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
  -H "Content-Type: application/json" \
  -d '{
    "data_inicio": "2026-01-01",
    "data_fim": "2026-04-30"
  }' \
  | python3 -m json.tool
```

9. Validação dos Endpoints com curl

9.1. GET /api/ranking/ — Ranking paginado

```
# Pagina 1 com 5 resultados
curl -s "http://localhost:8000/api/ranking/?page=1&page_size=5"
   \
  | python3 -m json.tool
```


```
{
    "count": 6195,
    "next": "http://localhost:8000/api/ranking/?page=2&page_size=5",
    "previous": null,
    "results": [
        {
            "id": 1,
            "nome": "BARRA/BA/CERRADO",
            "estado": "BA",
            "bioma": "CERRADO",
            "score_topsis": 0.5209,
            "ranking": 1,
            "nivel_risco": "CRITICO",
            "nivel_risco_display": "Critico",
            "total_focos": 1247,
            "frp_media": 82.35,
            "risco_historico_medio": 0.874,
            "dias_sem_chuva_medio": 42.1,
            "precipitacao_media": 0.0,
            "periodo_inicio": "2025-01-01",
            "periodo_fim": "2026-04-30"
        }
    ]
}
```


```
# Filtro por nivel de risco CRITICO
curl -s "http://localhost:8000/api/ranking/?nivel_risco=CRITICO"
    \
  | python3 -m json.tool

# Filtro por estado
curl -s "http://localhost:8000/api/ranking/?estado=BA" \
  | python3 -m json.tool

# Filtro por bioma
```

```
curl -s "http://localhost:8000/api/ranking/?bioma=CERRADO" \
  | python3 -m json.tool

# Combinado
curl -s "http://localhost:8000/api/ranking/\
?estado=MT&bioma=AMAZONIA&nivel_risco=ALTO" \
  | python3 -m json.tool
```

9.2. GET /api/focos/geojson/ — GeoJSON dos focos

```
# GeoJSON com filtro de estado (maximo 5000 features)
curl -s "http://localhost:8000/api/focos/geojson/?estado=AL" \
  | python3 -m json.tool | head -40

# Com bounding box (lon_min,lat_min,lon_max,lat_max)
curl -s "http://localhost:8000/api/focos/geojson/\
?bbox=-48.5,-13.5,-44.0,-10.0" \
  | python3 -m json.tool | head -30
```

9.3. GET /api/serie-temporal/ — Série mensal

```
curl -s "http://localhost:8000/api/serie-temporal/" \
  | python3 -m json.tool
```

```
[
    {"mes": "2025-01", "focos": 142831, "frp_medio": 28.4},
    {"mes": "2025-02", "focos": 98234,  "frp_medio": 25.1},
    {"mes": "2025-09", "focos": 612475, "frp_medio": 42.8},
    {"mes": "2026-04", "focos": 64221,  "frp_medio": 31.2}
]
```

9.4. GET /api/graficos-dados/ — Dados dos gráficos

```
# Todos os datasets para os 8 graficos em 1 chamada
curl -s "http://localhost:8000/api/graficos-dados/" \
  | python3 -m json.tool | head -60

# Saida contem: por_bioma, serie_temporal, top10,
# top5_radar, scores_por_nivel, scatter
```

9.5. GET /api/correlacao/ — Matriz de correlação

```
curl -s "http://localhost:8000/api/correlacao/" \
  | python3 -m json.tool
```

```
{
    "rotulos": [
        "C1 - Focos", "C2 - FRP",
        "C3 - Risco Hist.", "C4 - Dias s/ Chuva",
        "C5 - Precipitacao"
    ],
```

```
    "pearson": [
        [1.0,    0.421, 0.318, 0.287, -0.195],
        [0.421,  1.0,   0.254, 0.201, -0.143],
        [0.318,  0.254, 1.0,   0.387, -0.211],
        [0.287,  0.201, 0.387, 1.0,   -0.302],
        [-0.195,-0.143,-0.211,-0.302,  1.0  ]
    ],
    "pvalores": [
        [0.0, 0.0001, 0.0012, 0.0031, 0.0089],
        ...
    ],
    "n": 6195
}
```

9.6. GET /api/areas-risco/geojson/ — GeoJSON das áreas

```
# Todas as areas criticas como GeoJSON (Points)
curl -s \
  "http://localhost:8000/api/areas-risco/geojson/\
?nivel_risco=CRITICO" \
  | python3 -m json.tool | head -50
```

10. Frontend — Vue 3

10.1. Acessar a interface

Após os containers estarem rodando, acesse no navegador:

Interface web: `http://localhost:5173`

API Django: `http://localhost:8000/api/`

Admin Django: `http://localhost:8000/admin/`

10.2. Desenvolvimento frontend local (sem Docker)

```
cd frontend

# Instala dependencias Node.js
npm install

# Inicia servidor de desenvolvimento Vite
npm run dev

# Acessa em http://localhost:5173
# Hot-reload ativo: salvar um arquivo .vue atualiza o browser
```

10.3. Build de produção

```
cd frontend
```

```
# Gera os arquivos estaticos para producao
npm run build

# Os arquivos ficam em frontend/dist/
ls -lh dist/
```

11. Testes Automatizados

Passo 13 — Executar a suite completa de testes

```
# Suite completa: 46 testes (requer containers rodando)
docker compose exec backend \
  python manage.py test queimadas -v 2

# Saida esperada:
# Found 46 test(s).
# ...
# Ran 46 tests in 0.330s
# OK
```

Passo 14 — Executar apenas testes de sensibilidade (sem Docker)

```
# Na maquina local, com venv ativo
source backend/.venv/bin/activate
cd backend

# Testes OAT (SimpleTestCase - nao precisa de banco)
python manage.py test queimadas.test_sensibilidade -v 2

# Saida esperada:
# Found 5 test(s).
# Skipping setup of unused database(s): default.
# ...
# Ran 5 tests in 0.005s
# OK
```

11.1. Testes com cobertura de código

```
docker compose exec backend bash -c \
  "pip install coverage -q && \
   coverage run manage.py test queimadas && \
   coverage report -m"

# Saida esperada (trecho):
# Name                                Stmts   Miss  Cover
# ----------------------------------------------------
```

```
# queimadas/topsis_fuzzy.py        142      0   100%
# queimadas/views.py               210     18    91%
# queimadas/models.py               45      2    96%
# ---------------------------------------------------
# TOTAL                            420     22    95%
```

12. Documentação pdoc

```
# Dentro do container backend
docker compose exec backend bash

# Instala pdoc e gera documentacao
pip install pdoc
pdoc queimadas --output-dir docs/

# Sai do container
exit

# Copia a documentacao para a maquina local
docker compose cp backend:/app/docs ./backend/docs

# Abre no navegador
xdg-open backend/docs/queimadas/topsis_fuzzy.html
```

13. Comandos de Manutenção

13.1. Banco de dados

```
# Criar novas migracoes apos alterar models.py
docker compose exec backend \
  python manage.py makemigrations queimadas

# Aplicar migracoes
docker compose exec backend \
  python manage.py migrate

# Acessar psql interativo
docker compose exec db psql -U postgres -d queimadas

# Contar focos no banco
docker compose exec db psql -U postgres -d queimadas \
  -c "SELECT COUNT(*) FROM queimadas_focoqueimada;"

# Contar areas ranqueadas
docker compose exec db psql -U postgres -d queimadas \
  -c "SELECT nivel_risco, COUNT(*) \
```

```
      FROM queimadas_arearisco \
      GROUP BY nivel_risco \
      ORDER BY COUNT(*) DESC;"

# Backup do banco
docker compose exec db pg_dump \
  -U postgres queimadas > backup_$(date +%Y%m%d).sql

# Restaurar backup
docker compose exec -T db psql \
  -U postgres queimadas < backup_20260507.sql
```

13.2. Git — Fluxo de trabalho

```
# Cria nova branch para feature
git checkout -b feat/nova-funcionalidade

# Verifica status
git status

# Adiciona arquivos especificos
git add backend/queimadas/views.py
git add frontend/src/components/GraficosView.vue

# Commit com mensagem padrao Conventional Commits
git commit -m "feat(graficos): adiciona grafico de dispersao
   score x focos"

# Push da branch
git push origin feat/nova-funcionalidade

# Merge na main apos aprovacao
git checkout main
git merge feat/nova-funcionalidade
git push origin main
```

13.3. Limpeza do ambiente Docker

```
# Para todos os containers mantendo os dados
docker compose down

# Para e remove TUDO (containers + volumes + dados)
docker compose down -v

# Remove imagens nao utilizadas
docker image prune -f

# Remove volumes nao utilizados
docker volume prune -f
```

```
# Limpeza completa (usar com cuidado)
docker system prune -a -f
```

14. Resolução de Problemas Comuns

14.1. Container db não inicia

```
# Verifica os logs do banco
docker compose logs db

# Erro comum: porta 5432 ja em uso
sudo ss -tlnp | grep 5432

# Solucao: muda a porta externa no docker-compose.yml
# ports: ["5434:5432"]  <- usa 5434 externamente
```

14.2. PostGIS não disponível

```
# Verifica se a extensao PostGIS esta ativa
docker compose exec db psql -U postgres -d queimadas \
  -c "SELECT * FROM pg_extension WHERE extname='postgis';"

# Se vazia, habilita manualmente
docker compose exec db psql -U postgres -d queimadas \
  -c "CREATE EXTENSION IF NOT EXISTS postgis;"
```

14.3. Erro de importação circular no Django (test.py)

```
# NUNCA nomeie um arquivo test.py na raiz do app
# O Python carrega antes do django.test causando conflito

# Estrutura INCORRETA:
# queimadas/test.py        <- PROBLEMA

# Estrutura CORRETA:
# queimadas/tests.py          <- testes de integracao
# queimadas/test_sensibilidade.py  <- testes OAT
```

14.4. Frontend não conecta ao backend

```
# Verifica configuracao do CORS no Django (settings.py)
# deve conter:
# CORS_ALLOWED_ORIGINS = ["http://localhost:5173"]

# Verifica a variavel no frontend
cat frontend/.env
# VITE_API_BASE_URL=http://localhost:8000/api
```

```
# Testa o backend diretamente
curl http://localhost:8000/api/estatisticas/
```

14.5. Celery worker não processa tasks

```
# Verifica conexao com Redis
docker compose exec backend python3 -c \
  "import redis; r=redis.from_url('redis://redis:6379/0'); \
   print('Redis OK:', r.ping())"

# Reinicia o worker
docker compose restart worker

# Verifica logs do worker
docker compose logs worker --tail=50
```

15. Checklist Final — Sistema Rodando

Execute este checklist após seguir todos os passos anteriores para confirmar que o sistema está funcionando corretamente:

```
echo "=== CHECKLIST IGNISGEO ==="

echo -n "1. Containers rodando: "
docker compose ps --status=running | grep -c "running" && \
  echo "OK (esperado: 5)" || echo "FALHOU"

echo -n "2. API respondendo: "
curl -s http://localhost:8000/api/estatisticas/ \
  | grep -q "total_focos" && echo "OK" || echo "FALHOU"

echo -n "3. Frontend acessivel: "
curl -s http://localhost:5173 | grep -q "IgnisGeo" \
  && echo "OK" || echo "FALHOU"

echo -n "4. Focos importados: "
curl -s http://localhost:8000/api/estatisticas/ \
  | python3 -c "import sys,json; \
    d=json.load(sys.stdin); \
    print(f'{d[\"total_focos\"]:,} focos')"

echo -n "5. Ranking disponivel: "
curl -s "http://localhost:8000/api/ranking/?page_size=1" \
  | python3 -c "import sys,json; \
    d=json.load(sys.stdin); \
    print(f'{d[\"count\"]} areas ranqueadas')"
```

```
echo -n "6. PostGIS ativo: "
docker compose exec db psql -U postgres -d queimadas -tAc \
  "SELECT postgis_version();" 2>/dev/null \
  | head -1 && echo "OK" || echo "FALHOU"

echo "=== FIM DO CHECKLIST ==="
```

Sistema operacional quando todos os itens retornam OK:

```
1.  Containers rodando:  5 OK
2.  API respondendo:  OK
3.  Frontend acessivel:  OK
4.  Focos importados:  3.876.956 focos
5.  Ranking disponivel:  6195 areas ranqueadas
6.  PostGIS ativo:  3.4 USE_GEOS=1 ...  OK
```

16. Resumo dos Endereços Locais

Tabela 1: Endereços e portas do IgnisGeo em desenvolvimento local.

Serviço	Endereço	Descrição
Frontend	`http://localhost:5173`	Interface web Vue 3
API Django	`http://localhost:8000/api/`	Todos os endpoints REST
Admin	`http://localhost:8000/admin/`	Painel administrativo Django
PostgreSQL	`localhost:5433`	Acesso externo ao banco
Redis	`localhost:6379`	Broker Celery

Referências

[1] INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS (INPE). **BDQueimadas — Banco de Dados de Queimadas**. Disponível em: `https://queimadas.dgi.inpe.br`. Acesso em: abr. 2026.

[2] DOCKER INC. **Docker Documentation**. Disponível em: `https://docs.docker.com`. Acesso em: abr. 2026.

[3] POSTGIS DEVELOPMENT TEAM. **PostGIS 3.4 Documentation**. Disponível em: `https://postgis.net/docs`. Acesso em: abr. 2026.

[4] CELERY PROJECT. **Celery 5 Documentation**. Disponível em: `https://docs.celeryq.dev`. Acesso em: abr. 2026.

[5] CERQUEIRA, R. **IgnisGeo — Repositório**. Disponível em: `https://github.com/rogeriocerqueira/ignisGeo`. Acesso em: mai. 2026.